## DL.AddComment

**Basic Operation:** Appends user-defined text into the Docklight communication data window and/or active log file. It is primarily used to annotate test results or print debugging information.

**Syntax & Parameters:**

DL.AddComment [comment], [timeStampAfterComment], [lineBreakAndPadding]

| Parameter | Type | Description |
|---|---|---|
| **comment** | String | *Optional.* The text to display. Leaving this empty creates a blank line. |
| **timeStampAfterComment** | Boolean | *Optional.* **True** adds a timestamp before the next incoming character. **False** (Default) adds nothing. |
| **lineBreakAndPadding** | Boolean | *Optional.* **True** (Default) formats the output with line breaks and spaces. **False** prints the raw string exactly as provided. |

**Return Values:** Void

**Practical Example:**

VBScript

```
None


DL.AddComment "--- Commencing Sub-Test A ---"
```

---

## DL.ClearCommWindows

**Basic Operation:** Wipes all currently displayed data from the Docklight communication windows (Channels 1 and 2). Useful for resetting the view before starting a new test iteration.

**Syntax & Parameters:**

`DL.ClearCommWindows`

*(No parameters required)*

**Return Values:** Void

**Practical Example:**

VBScript

```
None


DL.ClearCommWindows
DL.AddComment "Screen cleared. Ready for new data."
```

---

## DL.GetDocklightTimeStamp

**Basic Operation:** Fetches the current date/time stamp using Docklight's internal formatting settings. It is often paired with `DL.AddComment` for custom logging.

**Syntax & Parameters:**

`result = DL.GetDocklightTimeStamp([myDateTime], [milliseconds], [trimmed])`

| Parameter | Type | Description |
|---|---|---|
| **myDateTime** | Date | *Optional.* A VBScript Date variable. `0` (Default) uses Docklight's internal time base. |
| **milliseconds** | Integer | *Optional.* Milliseconds (0-999). `-1` (Default) uses internal time base. `-2` omits milliseconds entirely. |

| | | |
|---|---|---|
| **trimmed** | Boolean | *Optional.* **True** removes the trailing space. **False** (Default) leaves the trailing space for compatibility. |

**Return Values:** String (The formatted timestamp).

**Practical Example:**

VBScript

```
None


timestamp = DL.GetDocklightTimeStamp(0, -1, True)
DL.AddComment "Event triggered at: " & timestamp
```

---

## DL.GetReceiveCounter

**Basic Operation:** Retrieves the number of times a specific "Receive Sequence" has been detected in the incoming data stream since communication started (or since the counter was last reset).

**Syntax & Parameters:**

result = DL.GetReceiveCounter(nameOrIndex)

| Parameter | Type | Description |
|---|---|---|
| **nameOrIndex** | String / Int | *Required.* The exact Name (String) or Sequence Index (Integer) of the Receive Sequence you want to check. |

**Return Values:** Long (The total hit count).

**Practical Example:**

VBScript

```
None


hits = DL.GetReceiveCounter("ErrorMsg")
```

```
If hits > 0 Then
    DL.AddComment "Warning: Errors detected!"
End If
```

---

## DL.OpenProject

**Basic Operation:** Loads an existing Docklight Project file (.ptp). This is heavily used in automated testing environments to switch between different communication setups or test suites dynamically.

**Syntax & Parameters:**

DL.OpenProject filePathName

| Parameter | Type | Description |
|---|---|---|
| **filePathName** | String | *Required.* The absolute or relative path to the .dck file. If left entirely blank (""), Docklight opens a file prompt dialog for the user. |

**Return Values:** Void

**Practical Example:**

VBScript

```
None


' Load a specific test project from the C: drive
DL.OpenProject "C:\TestProjects\DeviceDiagnostics.dck"
```

---

## DL.Pause

**Basic Operation:** Suspends the execution of your script for a specified duration in milliseconds. *Note: Docklight does not process OnReceive events while paused.*

**Syntax & Parameters:**

DL.Pause milliseconds

| Parameter | Type | Description |
| --- | --- | --- |
| **milliseconds** | Long | *Required.* The delay duration in milliseconds. |

**Return Values:** Void

**Practical Example:**

VBScript

```
None


DL.SendSequence "RebootCommand"
DL.Pause 2000 ' Wait 2 seconds for the device to reboot
DL.SendSequence "GetStatus"
```

---

## DL.Quit

**Basic Operation:** Terminates the script and completely exits the Docklight application. This is typically the final command in an unattended, automated test script.

**Syntax & Parameters:**

DL.Quit

*(No parameters required)*

**Return Values:** Void

**Practical Example:**

VBScript

```
None


DL.StopLogging
DL.AddComment "Testing complete. Shutting down."
DL.Quit
```

---

## DL.ResetReceiveCounter

**Basic Operation:** Resets the hit counter to zero for a specific Receive Sequence. If called without parameters, it resets the counters and character-matching buffers for *all* Receive Sequences.

**Syntax & Parameters:**

DL.ResetReceiveCounter [nameOrIndex]

| Parameter | Type | Description |
| --- | --- | --- |
| **nameOrIndex** | String / Int | *Optional.* The Name or Sequence Index of a specific Receive Sequence to reset. Omit to reset all. |

**Return Values:** Void

**Practical Example:**

VBScript

```
None


' Reset all counters before sending a command to ensure a fresh read
DL.ResetReceiveCounter
DL.SendSequence "Ping"
```

---

## DL.SendSequence

**Basic Operation:** Transmits a defined "Send Sequence" from your project list out through the active communication port.

**Syntax & Parameters:**

DL.SendSequence nameOrIndex, [parameters], [representation]

| Parameter | Type | Description |
| --- | --- | --- |

| | | |
|---|---|---|
| **nameOrIndex** | String / Int | *Required.* The Name or Sequence Index of the Send Sequence. |
| **parameters** | String | *Optional.* Comma-separated values to replace wildcard regions (e.g., `<#>`) in the sequence. |
| **representation** | String | *Optional.* Data format of parameters: `"A"` (ASCII), `"H"` (Hex), `"D"` (Decimal), or `"B"` (Binary). |

**Return Values:** Void

**Practical Example:**

VBScript

```
None


' Send Sequence at index 0 and inject a Hex parameter
DL.SendSequence 0, "FF", "H"
```

---

## DL.StartCommunication

**Basic Operation:** Opens the serial ports (or network sockets) as configured in your project settings and starts capturing data.

**Syntax & Parameters:**

DL.StartCommunication

*(No parameters required)*

**Return Values:** Void

**Practical Example:**

VBScript

```
None
```

```
DL.StartCommunication
DL.AddComment "Ports are now open."
```

---

## DL.StopCommunication

**Basic Operation:** Closes the active communication ports, stopping all data transmission and reception.

**Syntax & Parameters:**

DL.StopCommunication

*(No parameters required)*

**Return Values:** Void

**Practical Example:**

VBScript

```
None


DL.StopCommunication
DL.AddComment "Ports closed safely."
```

---

## DL.StartLogging

**Basic Operation:** Starts recording incoming and outgoing data to a file. It can output simultaneously in multiple formats (ASCII, Hex, Decimal, Binary).

**Syntax & Parameters:**

DL.StartLogging baseFilePath, [appendData], [representations], [format], [highspeed], [noHeaders]

| Parameter | Type | Description |
| --- | --- | --- |

| | | |
|---|---|---|
| **baseFilePath** | String | *Required.* The directory and base filename for the log file(s). |
| **appendData** | Boolean | *Optional.* **True** (Default) appends to existing files. **False** overwrites them. |
| **representations** | String | *Optional.* Characters denoting the file types to create: `"A"`, `"H"`, `"D"`, `"B"`. Default is `"AHDB"` (creates all four). |
| **format** | Integer | *Optional.* **0** (Default) creates standard `.txt`. **1** creates HTML. **2** creates RTF. |
| **highspeed** | Boolean | *Optional.* **True** disables the visual communication window to save CPU on fast data streams. **False** (Default) keeps the window active. |
| **noHeaders** | Boolean | *Optional.* **True** prevents the "Docklight Log File started" text from printing in the file. **False** (Default) includes it. |

**Return Values:** Void

**Practical Example:**

VBScript

None

```
' Start logging in ASCII and Hex formats, overwriting old files
DL.StartLogging "C:\Logs\TestRun", False, "AH"
```

---

## DL.StopLogging

**Basic Operation:** Halts any active logging session and cleanly closes the log files on your hard drive.

**Syntax & Parameters:**

DL.StopLogging

*(No parameters required)*

**Return Values:** Void

**Practical Example:**

VBScript

```
None


DL.StopLogging
DL.AddComment "Logging stopped and file saved."
```

---

## DL.WaitForSequence

**Basic Operation:** Halts the script and waits until a specific Receive Sequence is detected in the data stream a certain number of times, or until a timeout occurs. *This is critical for query/response testing.*

**Syntax & Parameters:**

result = DL.WaitForSequence(nameOrIndex, [maxCounter], [timeout])

| Parameter | Type | Description |
|---|---|---|
| **nameOrIndex** | String / Int | *Required.* The Name or Sequence Index of the Receive Sequence to monitor. |
| **maxCounter** | Long | *Optional.* The target hit count. Default is **1**. Use **-1** to ignore the counter and wait solely for the timeout duration. |

| | | |
|---|---|---|
| **timeout** | Long | *Optional.* Timeout limit in milliseconds. Default is **-1** (wait indefinitely). |

**Return Values:** Long (The number of times the sequence was detected before returning).

**Practical Example:**

VBScript

```
None


DL.ResetReceiveCounter
DL.SendSequence "RequestData"

' Wait up to 3000ms for 1 occurrence of "DataResponse"
hits = DL.WaitForSequence("DataResponse", 1, 3000)

If hits = 1 Then
    DL.AddComment "Response received in time!"
Else
    DL.AddComment "Error: Device timeout."
End If
```